TU/CSE

TEZPUR UNIVERSITY

End Semester Examination, Spring 2023

CO 314 System Software and Compiler Design

Marks : 60 *Time : 3 hours*

All questions are compulsory. The figures in the right-hand margin indicate marks for the individual question.

---

1. (a) What are the different types of assembly language statements from an assembler's perspective? Give examples. 3

   (b) What is *forward reference* in an assembly language program. How is it handled in a two-pass assembler? 1 + 2

2. Write a context free grammar G for simple assignment statements involving scalar variables and constants for a language like C. Draw a parse tree for the string $x = 3 * y + 2$ using grammar G 2 + 2

3. (a) Why should *back-tracking* be avoided in top-down parsing? Using an example show how back-tracking can be avoided. 2 + 2

   OR

   (b) What are the conditions that imply that a grammar is not LL(1)? 4

4. What are two types of conflicts possible in preparing an LR parser? What criteria are used to avoid conflicts in building SLR(1) and CLR(1) parsers? 2 + 3

5. (a) Describe *three address code* as used as a format for intermediate code generated by a compiler. 3

   (b) Represent the following expression in syntax-tree form- 2

$$x = a + b * c$$

6. Write semantic actions to produce postfix code for input corresponding to the grammar rule- 2

$$E \rightarrow E + T$$

7. Why are function parameters and local variables located in *activation records?* 2

8. (a) Why is implementation of symbol table more complicated for a block-structured language than that for an assembly language. 5

(b) During compilation new symbols are to be entered in the symbol table, existing symbols are to be removed, and attributes of specific symbols accessed from the symbol table. Discuss how these requirements determine the data structure that are suitable for a symbol table. 3

9. Why does a compiler do error handling? 2

10. Consider the following three-address code sequence, S-

```
(1)  if n < 10 goto (3)
(2)  goto (10)
(3)  T2 = n + 1
(4)  n = T2
(5)  T3 = n * 4
(6)  T4 = addr(s)[T3]
(7)  T5 = n * 4
(8)  addr(x)[T5] = T4
(9)  goto (1)
```

(a) Identify the basic blocks. 2

(b) Draw the DAG for the code portion from (3)-(9). 4

(c) Prepare the revised quadruple sequence from the DAG. 3

11. Using example cases show how *global data flow analysis* can help in code optimisation. 5

12. A simple way to produce machine code from three-address code is to produce a pre-determined machine instruction

for a each particular three-address code operation and operands. What are two ways in which "better" machine code can be generated from the three-address code? 4

13. (a) How is static linking different from dynamic linking? 4

OR

(b) How can *program relocation* be performed? 4